Indu Singh, AP, CSE, DTU

COPY CONSTRUCTOR

A copy constructor takes an object of the class as an argument & copies data values of members of one object into the values of members of another object. Since it takes only 1 argument, it is also known as one argument constructor. The primary use of a copy constructor is to create a new object from an existing one by initialization.
For this, the copy constructor takes a reference to an object of the same class as an argument.

Conditions where copy constructor is used —

(1) When an object is created & initialized with another object of same class.

```
student S2(S1);
```

(2) When an object is created & another object of same class is equated to it.

```
student S2 = S1;
```

(3) When a function is called by using call by value method & object is passed as parameter.

```
T2.add(T1);
```

(4) When a function returns an object.

```
Student S1, S2;
Student S2 = func();
```

PROGRAM OF COPY CONSTRUCTOR

```
class Student
{   int RollNo;
    char name[30];
    float per;
public:
    Student()
    {   rollno = 0;
        strcpy(name, "not assigned");
        per = 0;
    }
Student(int rno, char n[], float p)
{   rollno = rno;
    strcpy(name, n);
    per = p;
}
```

parameterized constructor

```
// student ( student &s)  //copy
                          constructor
{
    rollno = s.rollno;
    strcpy (name , s.name);
    per = s.per;
}

student getstudent ();
void display()
{  cout <<
   cout <<
   cout <<
}
};

student Student :: getstudent ()
{
    student S;
    cout << Enter Rollno ";
    cin >> S.rollno ;
    return (S);
}
```

```
void main()
{  Student S1;
   cout << " Data mbers values
            using default constructor ";
   S1.display()
   Student S2(2, "Amit", 80.2);
                  // parameterized constructor
                     called implicitly
cout << " Data mbers values using
                  parameterized constructor ";
S2.display();
Student S3(S2); // copy constructor
                     called implicitly
S1 = S3 ; // copy constructor called
                  automatically to assign one object
                     to another.
S1 = S2.getstudent(); // copy constructor
                  called automatically for func^n
                     returning object.
```

Program 10.3 Write a program that uses overloaded constructors and dynamically allocates memory to a string. Demonstrate the use of copy constructor.

```
#include<iostream.h>
#include<string.h>
class String
{   private:
        int length;
        char *str;
    public:
        String()
        {   length = 0;   }
        String(char *s)
        {   length = strlen(s);
            str = new char [length + 1];
            strcpy(str, s);
        }
        String(String &s)
        {   length = s.length;
            str = new char[length + 1];
            strcpy(str, s.str);
        }
        void show_str()
        {   cout.write(str, length); }
};
main()
{   String s1("Welcome to the world of programming");
    String s2 = s1;
    cout<<"\n String (s2) = ";
    s2.show_str();
}
```

OUTPUT

```
String (s2) = Welcome to the world of programming
```

10.5 DESTRUCTORS

Like a constructor, a destructor is also a member function that is automatically invoked. However, unlike the constructor which constructs the object, the job of destructor is to destroy the object. For this, it deallocates the memory dynamically allocated to the variable(s) or perform other clean up operations.

Calling a constructor, destructor

```
#include<iostream.h>
class Sample
{   private:
        int x;
    public:
        Sample(int n)
        {   x = n;
            cout<<"\n Constructor Called for object with value : "<<x;
        }
~Sample()
{   cout<<"\n Destructor Called for object with value : "<<x;
    }
};
main()
{   Sample S1(1);
    Sample S2(2);
    Sample S3(3);
  }
```

OUTPUT

```
Constructor Called for object with value : 1
Constructor Called for object with value : 2
Constructor Called for object with value : 3
Destructor Called for object with value : 3
Destructor Called for object with value : 2
Destructor Called for object with value : 1
```